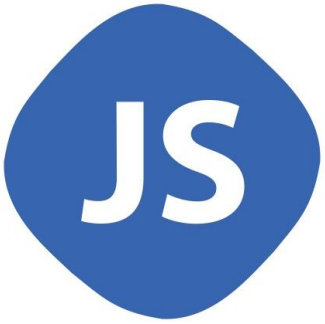


Коллекция Map

- Введение
- Коллекция Map
- Итерация коллекции Map
- Перебор коллекции Map
- Map и Object

Введение

Сейчас мы знаем о следующих сложных структурах данных:

- **Объекты** для хранения именованных коллекций.
- **Массивы** для хранения упорядоченных коллекций.

> ***Примечание***.
>
> **Массивы** нужно использовать тогда, когда имеет значение **последовательность** информации.
>
> **Объекты —** лучший выбор, если вы хотите структурировать информацию по **названию данных**.

Вообще, в массивах и объектах можно хранить разные структуры данных. Для лучшей **масштабируемости кода** вам практически всегда нужна будет комбинация из нескольких структур:

- массивы внутри объектов;
- объекты внутри массивов;
- массивы внутри массивов;
- объекты внутри объектов.



Как обычно, все зависит от поставленных программисту задач и предметной области. Но, выбирая структуру данных, помните, **объекты внутри внешнего массива** – классическая структура реляционных баз данных.

Но этого не всегда достаточно для решения повседневных задач. Поэтому также существуют **Map** и **Set**.

Коллекция Map

Map – это коллекция ключ/значение, как и Object. Но основное отличие в том, что **Map** позволяет использовать ключи любого типа.

Методы и **свойства**:

- new **Map**() – создаёт коллекцию.
- map.**set**(key, value) – записывает по ключу **key** значение **value**.
- map.**get**(key) – возвращает значение по ключу или **undefined**, если ключ **key** отсутствует.
- map.**has**(key) – возвращает **true**, если ключ **key** присутствует в коллекции, иначе **false**.
- map.**delete**(key) – удаляет элемент (пару «ключ/значение») по ключу **key**.
- map.**clear**() – очищает коллекцию от всех элементов.
- map.**size** – возвращает текущее количество элементов.

Пример создания коллекции **Map** приведен в демонстрационном примере **example_1**.

Обратите внимание, в отличие от объектов, ключи коллекции не будут приведены к строкам. Можно использовать **любые типы данных** для ключей.

example_1. Создание коллекции Map

```
<script>

    // создаем объект коллекции Map
```

```
    let map = new Map();
    // устанавливаем объекту значения
    map.set(101, "number");  // число как ключ
    map.set(true, "boolean");  // булево значение как ключ
    map.set("1", "string");  // строка как ключ
    map.set(1, "digit");  // цифра как ключ
    // получаем доступ к значениям объекта
    console.log("map.get(101) =", map.get(101));  // number
    console.log("map.get(true) =", map.get(true)); // boolean
    // обычный объект Object приводит ключи к строкам
    // Map сохраняет тип ключей, так что в этом случае сохранится
    2 разных значения:
    console.log("map.get(1) =", map.get(1));  // digit
    console.log("map.get('1') =", map.get("1"));  // string
    console.log("----------");
    console.log("Размер коллекции: ", map.size); // 4
</script>
```

Сохраним в коллекцию данные о ноутбуке, в качестве ключей используем его атрибуты.

example_2. Сохранение в коллекцию объекта данных

```
<script>
    // создание объекта коллекции
    let map = new Map();
    // записываем в коллекцию данные ноутбука
    // используем классические строковые ключи
    map.set("id", 1);
```

```
    map.set("title", "Ноутбук HUAWEI MateBook D16 RLEF-X");
    map.set("gpu", "Intel Iris Xe Graphics");
    map.set("cpu", "Intel Core i5-12500H");
    map.set("cpu_cores", 12);
    map.set("ram", "16 ГБ");
    map.set("diagonal", "16'");
    map.set("price", 76.299);
    map.set("img", "huawei-matebook-d16-rlef-x.jpg");
    console.log("Коллекция Map:");
    // вывод коллекции в консоль
    console.log(map);
</script>
```

Лучше понимаешь структуру данных, когда создаешь её вручную.

В демонстрационных примерах **example_3** и **example_4** при ручной инициализации коллекции **Map** укажем **двумерный массив** с парами ключ-значение.

example_3. Создание коллекции вручную

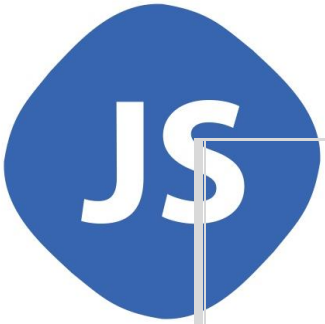
```
<script>
    // ручное создание объекта коллекции
    // инициируем коллекцию через создание двумерного массива
    let map = new Map([
        ["id", 1],
        ["title", "Ноутбук HUAWEI MateBook D16 RLEF-X"],
        ["gpu", "Intel Iris Xe Graphics"],
        ["cpu", "Intel Core i5-12500H"],
```

```
        ["cpu_cores", 12],
        ["ram", "16 ГБ"],
        ["diagonal", "16'"],
        ["price", 76.299],
        ["img", "huawei-matebook-d16-rlef-x.jpg"]
    ]);
    console.log("Коллекция Map:");
    // вывод коллекции в консоль
    console.log(map);
</script>
```

Как обычно, сложность создаваемой структуры данных определяется поставленными перед разработчиком задачами.

example_4. Создание вручную сложной коллекции

```
<script>
    // коллекция может быть сколько угодно сложной
    // от этого логика ее создания не меняется
    let map = new Map([
    [
        1,
        {
            id: 1,
            title: 'Ноутбук HUAWEI MateBook D16 RLEF-X',
            gpu: 'Intel Iris Xe Graphics',
            cpu: 'Intel Core i5-12500H',
            cpu_cores: 12,
```



```
            ram: '16 ГБ',
            diagonal: "16'",
            price: 76.299,
            img: "huawei-matebook-d16-rlef-x.jpg"
        }
    ],
    [
        2,
        {
            id: 2,
            title: 'Ультрабук HUAWEI MateBook D 15 BoM-WFQ9',
            gpu: 'AMD Radeon Graphics',
            cpu: 'AMD Ryzen 5 5500U',
            cpu_cores: 6,
            ram: '16 ГБ',
            diagonal: "15.6'",
            price: 62.999,
            img: "huawei-matebook-d-15bom-wfq9.jpg"
        }
    ]
    ]);
    console.log("Коллекция Map:");
    // вывод коллекции в консоль
    console.log(map);
</script>
```



Динамическое создание и вывод коллекции **Map**. Цикл **for** не лучшая конструкция для динамического вывода коллекций. Далее рассмотрим правильные способы, но если ключи представляют из себя непрерывную последовательность чисел, то почему бы и нет.

example_5. Динамическое создание коллекций

```
<script>
    // создаем объект коллекции
    let map = new Map();
    // проходим циклом по элементам массива laptop
    laptop.forEach((obj, index) => {
        // записываем массив в коллекцию
        map.set(index, obj);
    })
    // просмотр коллекции
    // console.log(map);
    // тестовый доступ
    // console.log(map.get(0).title);
    // выводим данные коллекции в браузер
    for (i=0; i < map.size; i++) {
        document.write(`
            ${map.get(i).id} <br>
            ${map.get(i).title} <br>
            ${map.get(i).gpu} <br>
            ${map.get(i).cpu} <br>
            ${map.get(i).cpu_cores} <br>
            ${map.get(i).ram} <br>
```

```
            ${map.get(i).diagonal} <br>
            ${map.get(i).price} <br>
            ${map.get(i).img} <br>
            <hr>
        `);
    }
</script>
```

Коллекция **Map** может использовать объекты в качестве ключей.

Использование объектов в качестве ключей – одна из наиболее заметных и важных функций **Map**. Это то, что невозможно для **Object**. Строка в качестве ключа в **Object** – это нормально, но мы не можем использовать другой **Object** в качестве ключа в **Object**.

example_6. Использование объектов в качестве ключей

```
<script>
    // создаем объекты
    let john = { name: "John" };
    let michael = { name: "Michael", login:"mich-mich" };
    // создаем коллекцию
    let person = new Map();
    // записываем в коллекцию значения
    // в качестве ключей используем объекты
    person.set(john, 123);
    person.set(michael, "moderator");
    // доступ к значению коллекции по ключу-объекту
    console.log(person.get(john)); // 123
    console.log(person.get(michael)); // moderator
```

```
</script>
```

Объединим в коллекцию данные двух **связанных структур**. В качестве **ключа** используем один объект, в качестве **значения** связанный с ним объект другого массива.

example_7. Соединение в коллекцию связанных объектов

```
<script>
    // массив педагогов
    let personnel = Array (
    {
        'id_personnel' : 11,
        'surname' : 'Бородина',
        'name' : 'Ксения',
        'patronymic' : 'Алексеевна'
    },
        { //... },
        { //... },
        { //... },
        { //... }
    );
    // создаем объект Map
    let personMap = new Map();
    // выполняем итерацию по объектам массива personnel
    personnel.forEach( person => {
        // для каждого преподавателя ищем соответствие в массиве
        data
        dataPerson = data.find( data => {
```

```
            return person.id_personnel == data.id_personnel;
        })
        // запишем в коллекцию соответствующие другу другу
        объекты разных массивов
        // объект из personnel - ключ
        // объект из data - значение ключа
        personMap.set(person, dataPerson);
    });
    // т.о. объединили данные разных массивов в одну коллекцию
    // вывод в консоль результирующей коллекции
    console.log(personMap);
</script>
```

Каждый вызов **map.set** возвращает объект **Map**, так что мы можем объединить вызовы в цепочку.

> ***Примечание***. Такой способ объединения программисты часто называют – **чейнить**. Чейнить - от слова *to chain* (англ. "выстраивать в цепочку").

example_8. Чейним коллекцию на методе set()

```
<script>
    // создание объекта коллекции
    let note = new Map();
    // цепочка вызовов метода set
    note
        .set("id", 1)
        .set("title", "Ноутбук HUAWEI MateBook D16 RLEF-X")
```

```
        .set("gpu", "Intel Iris Xe Graphics")
        .set("cpu", "Intel Core i5-12500H")
        .set("cpu_cores", 12)
        .set("ram", "16 ГБ")
        .set("diagonal", "16'")
        .set("price", 76.299)
        .set("img", "huawei-matebook-d16-rlef-x.jpg");
    // вывод коллекции в консоль
    console.log(note);
</script>
```

Итерация коллекции Map

Как обычно, сложные структуры данных хороши тогда, когда есть способ обойти их значения в цикле.

Итерация коллекции **специальным** циклом **for...of**.

example_9. Итерация коллекции через for...of

```
<script>
    // перебор коллекции Map специальным циклом of
    // каждый элемент коллекции - массив
    for ( item of courses_map ) {
        console.log("key: " + item[0], item[1]);
    };
    console.log("===============");
    // или так ...
```

```
    // альтернативная форма записи
    for ( [key, elem] of courses_map ) {
        console.log("key: " + key, elem);
    };
</script>
```

Использование для итерации встроенного метода **forEach**.

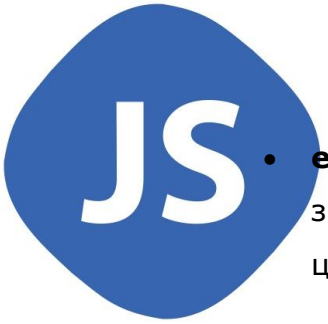
example_10. Итерация коллекции через forEach

```
<script>
    // перебираем коллекцию Map циклом forEach
    // в теле цикла к коллекции доступ через синоним - map
    courses_map.forEach( (item, key, map) => {
        console.log(key);
        console.log(item);
        // или так...
        // console.log(map.get(key));
        console.log("---------------");
    });
</script>
```

Перебор коллекции Map

Для перебора коллекции **Map** есть три **метода**:

- **keys**() – возвращает итерируемый объект по ключам,
- **values**() – возвращает итерируемый объект по значениям,



- **entries**() – возвращает итерируемый объект по парам вида [ключ, значение], этот вариант используется по умолчанию в специальном цикле **for..of**.

> ***Примечание***. В отличие от обычных объектов Object, в **Map** перебор происходит в том же порядке, в каком происходило добавление элементов.

Метод keys()

Метод **keys**() возвращает итерируемый объект по ключам.

example_11. Итерируем коллекцию по ключам

```
<script>

    // создаем объект коллекции

    let tel = new Map();

    tel

        .set("diag", "24")

        .set("matr", "LED")

        .set("freq", "60 Гц")

        .set("firm", "DEXP")

        .set("model", "H24F7000E")

        .set("perm", "HD")

        .set("price", "25000")

        .set("path", "img/H24F7000E.jpg")

        .set("descr", "Телевизор LED DEXP H24F7000E отличается
        столь компактными размерами, что место для него найдется
        даже на самой маленькой кухне. Благодаря VESA-креплению
        разместить его получится даже на стене, чтобы освободить
        на горизонтальных поверхностях место для других вещей.")
```



```
    // возвращает итерируемый объект - набор ключей коллекции
    console.log(tel.keys());
    console.log("---------------");
    // выполняем итерацию по ключам коллекции
    for (key of tel.keys()) {
        console.log(key);
    };
</script>
```

Усложним задачу, из подключаемой структуры данных создадим коллекцию и переберем ее по динамически сгенерированным ключам.

example_12. Перебираем массив объектов, создаем объект коллекции

```
<script>
    // создаем объект коллекции Map
    var tel = new Map();
    // динамическое создание коллекции Map
    // со случайно сгенерированным значением ключа
    televisions.forEach( (item, index) => {
        // генерируем значение ключа
        // максимум не включается, минимум включается
        let key = Math.floor(Math.random() * (1000 - 1)) + 1;
        // сохраняем в коллекцию пару ключ – значение, где:
        // -> ключ - id# + случайное число
        // -> значение - объект массива televisions
        tel.set("id#" + key, item);
    });
```

```
    // коллекция ключей
    // console.log(tel.keys());
    // итерация по ключам коллекции
    for (key of tel.keys()) {
    console.log(key);
    console.log(tel.get(key));
    console.log("===============");
    };
</script>
```

Метод values()

Метод **values**() – возвращает итерируемый объект по значениям.

example_13. Итерируем коллекцию по значениям

```
<script>
    // создаем коллекцию Map
    note = new Map();
    // заполняем коллекцию данными
    note.set("id", 1);
    note.set("title", "Ноутбук HUAWEI MateBook D16 RLEF-X");
    note.set("gpu", "Intel Iris Xe Graphics");
    note.set("cpu", "Intel Core i5-12500H");
    note.set("cpu_cores", 12);
    note.set("ram", "16 ГБ");
    note.set("diagonal", "16'");
    note.set("price", 76.299);
```

```
    note.set("img", "huawei-matebook-d16-rlef-x.jpg");
    // возвращает итерируемый объект - набор значений коллекции
    console.log(note.values());
    console.log("---------------");
    // итерация по строковым значениям коллекции
    // значение может быть сколько угодно сложным объектом
    for (val of note.values()) {
        console.log(val);
    };
</script>
```

Подключим к сценарию массив данных, теперь итерируемое значение – двумерный массив.

example_14. Итерируемое значение – двумерный массив

```
<script>
    // создание коллекции
    let map = new Map();
    // перебор элементов массива clothes
    clothes.forEach ( (item, index) => {
        // из массива создадим объект коллекции Map
        map.set(index, item);
    });
    // перебор значений коллекции
    for (item of map.values()) {
        document.write(`
        <h2>${item[1]} (код: ${item[0]})</h2>
            <img src='images/${item[8]}' width='150px'> <br>
            Бренд: <b> ${item[2]} </b><br>
            Цвет: <b> ${item[3]} </b><br>
            Страна: <b> ${item[4]} </b><br>
            Капюшон: <b> ${item[5]} </b><br>
```

```
            Размер: <b> ${item[6]} </b><br>
            Цена: <b> ${item[7]} </b><br>
            <hr>
        `);
    };
</script>
```

Метод entries()

Метод **entries**() – возвращает итерируемый объект по парам вида **ключ** – **значение**.

example_15. Итерируем коллекцию по паре ключ – значение

```
<script>
    // создаем объект коллекции Map
    var clothes = new Map();
    // чейним элементы коллекции
    clothes.
        set("id", "3").
        set("title", "Парка").
        set("brand", "Cap Horn").
        set("colour", "Зеленый").
        set("country", "Китай").
        set("hood", "Есть").
        set("size", "48").
        set("price", "14.590").
        set("image", "images/parka-cap.jpg");
    // возвращает итерируемый объект - набор пар ключ-значение
    console.log(clothes.entries());
```

```
    console.log("---------------");
    // перебираем пары ключ - значение коллекции
    for (arr of clothes.entries()) {
        console.log(arr[0], " -> ", arr[1]);
    };
</script>
```

Выполним еще один перебор в формате ключ – значение на примере подключаемой коллекции со сложной структурой данных.

example_16. Сложная структура данных не всегда сложный код

```
<script>
    // итерируем сложный массив данных - note
    for ( [key, item] of note.entries()) {
        console.log(key, " -> ", item.title);
    };
</script>
```

Map и Object

Map из Object

Создание коллекции **Map** из объекта.

Если у нас уже есть обычный объект, и мы хотели бы создать **Map** из него, то поможет встроенный метод **Object.entries**(obj), который получает объект и возвращает массив пар ключ-значение в формате, необходимом для создания коллекции.

Важно. В логической цепочке структур хранения и конструкций обработки данных **- Коллекции - Массивы - Циклы** не хватает одного очень важного звена - **Объекта**. Объектам будет посвящена отдельная тема.

example_17. Из объекта коллекцию в одну строку кода

```
<script>
    // создаем объект
    let tel = {
        diag: "24",
        matr: "LED",
        freq: "60 Гц",
        firm: "DEXP",
        model: "H24F7000E",
        perm: "HD",
        price: 25000,
        path: "img/H24F7000E.jpg",
        descr: "Телевизор LED DEXP H24F7000E отличается столь
        компактными размерами, что место для него найдется даже
        на самой маленькой кухне. Благодаря VESA-креплению
        разместить его получится даже на стене, чтобы освободить
        на горизонтальных поверхностях место для других вещей."
    };
    // создаем коллекцию из объекта
    myMap = new Map(Object.entries(tel));
    // вывод коллекции в консоль
    console.log(myMap);
    // тестовый доступ
```

```
    // console.log(myMap.get("descr"));
</script>
```

Object из Map

В JavaScript есть метод **Object.fromEntries**, который делает противоположное: получив массив пар вида [ключ, значение], он создаёт из них объект.

example_18. Создание объекта из коллекции

```
<script>
    // создание объекта из коллекции Map
    televisions_obj =
    Object.fromEntries(televisions_map.entries());
    // исходная коллекция
    console.log(televisions_map);
    // результирующий объект
    console.log(televisions_obj);
</script>
```

В **самостоятельных работах** к уроку рассмотрим:

- Создание объектов коллекции Map вручную.
- Динамическое создание объектов коллекции Map из сложных структур данных.
- Создание объекта коллекции Map из элементов DOM-документа.
- Вывод объекта коллекции Map в документ.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы